

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express



High level Design – E-Commerce Microservices Architecture

Project Overview

The **E-commerce Microservices Application** is designed to provide a scalable and efficient architecture using **Spring Boot, Java, and REST APIs**. The system is containerized using **Docker** and orchestrated with **Kubernetes** to ensure high availability and scalability. Various **design patterns** have been implemented to enhance modularity, maintainability, and fault tolerance.

Technologies Used

- **Backend:** Spring Boot, Java, REST API
- **Containerization & Orchestration:** Docker, Kubernetes
- **Service Discovery:** Eureka Server
- **Communication:** Feign Client, Circuit Breaker (Resilience4J)
- **Security:** JWT Authentication, Proxy Client
- **API Management:** API Gateway
- **Monitoring :** Prometheus & Grafana
- **Databases :** MYSQL & MongoDB
- **Event Driven :** Apache Kafka

Key Outcomes of the Course

- Build and deploy a corporate-level e-commerce platform with microservices.
- Gain hands-on experience with Eureka, Feign Client, Circuit Breaker, and JWT at gateway layer.
- Learn Kubernetes deployments and PVC configurations for microservices.
- Monitor and troubleshoot microservices using Prometheus, Grafana, and Zipkin.
- Master end-to-end architecture for scalable, production-ready systems.
- Secure your microservices with best practices for authentication and authorization.
- Confidence in Building Real-World Applications
- End-to-End Development Skills

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

Microservices Components

1. User Microservice
2. Product Microservice
3. Order Microservice
4. Payment Microservice
5. Favourite Microservice
6. Shipping Microservice
7. Audit Microservice
8. **Eureka Server** - Service Discovery for Microservices Architecture
9. **Feign Client** Communication for All Microservices
10. **API Gateway** for E-commerce Microservices
11. **Circuit Breaker** Implementation in API Gateway
12. Distributed Tracing with **Zipkin** in Microservices
13. Monitoring E-Commerce Microservices with **Prometheus & Grafana**
14. Dockerizing Microservices for E-Commerce Application
15. Deploying Microservices on **Kubernetes & Database Persistence**
16. Proxy Client Service
17. Final Project Review and Deployment

User Microservice

The **User Microservice** is a core component of the **E-commerce Microservices Application**, responsible for managing user-related functionalities such as registration, authentication, and CRUD operations for user data, addresses, and credentials.

The **User Microservice** provides the following functionalities:

1. **User Registration**
 - Users can register with basic details:
 - userId
 - firstName
 - lastName
 - email
 - phone
2. **Address Management**
 - A user can have **multiple addresses** (One-to-Many Relationship).

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

- Supports **Create, Read, Update, and Delete (CRUD)** operations.

3. Credentials Management

- Each user has a **one-to-one** relationship with credentials.
- The credentials include:
 - **Username**
 - **Password (hashed & secured)**
 - **Role-Based Authority (User/Admin)**

4. Role-Based Authorization

- Two roles are available:
 - **USER:** Can access and modify their own data.
 - **ADMIN:** Has additional privileges for managing users.
- Implements **JWT Authentication** for secure access.

5. CRUD Operations

- Users can perform **CRUD operations** on their profile, addresses, and credentials.

6. DTO Pattern Implementation

- Uses **DTOs (Data Transfer Objects)** to ensure that:
 - Only necessary data is exposed in API responses.
 - Request and response objects are well-structured.

Database Relationship

Entity	Relationship Type	Target Entity
User → Address	One-to-Many	A user can have multiple addresses
User → Credential	One-to-One	A user has one credential
Credential → User	One-to-One (Bidirectional)	Each credential belongs to one user
Address → User	Many-to-One	Each address belongs to a user
Credential → RoleBasedAuthority	Enum Mapping	Stores role as a string

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

@JsonIgnore

Entity	Field	Mapping Type	Annotation	Why Use @JsonIgnore?
User	addresses	@OneToMany (User → Address)	@JsonIgnore	Prevents infinite recursion because Address has a reference back to User .
User	credential	@OneToOne (User → Credential)	No @JsonIgnore	Credentials may need to be included in JSON response (unless sensitive).
Credential	user	@OneToOne (Credential → User)	@JsonIgnore	Prevents recursion since User already contains Credential .
Address	user	@ManyToOne (Address → User)	No @JsonIgnore	Keeping User details available in Address responses.

Why Use @JsonIgnore ?

- **Prevents Infinite Recursion** when User has a list of Address objects, and Address has a reference back to User .
- **Avoids Performance Issues** by not loading unnecessary nested objects.
- **Fixes Lazy Loading Errors** (LazyInitializationException) when using FetchType.LAZY .

API Endpoints

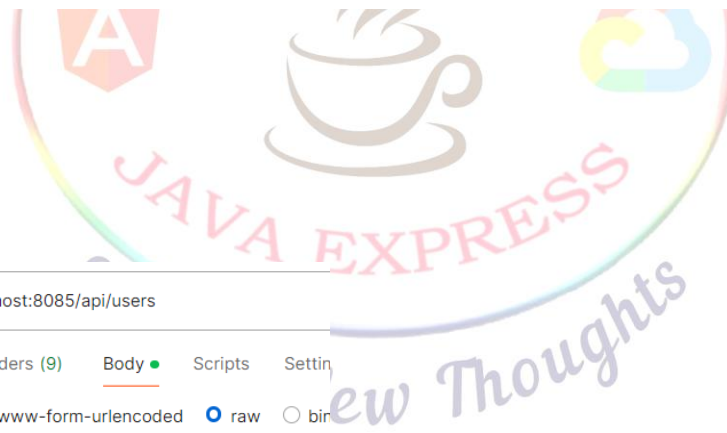
Create User

POST http://localhost:8085/api/users

Params Authorization Headers (9) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ bin

```
1 {
2   "firstName": "vijay",
3   "lastName": "John",
4   "email": "vijay12@gmail.com",
5   "phone": "9876543245",
6   "credential": {
7     "username": "vijay12",
8     "password": "123456",
9     "roleBasedAuthrority": "ROLE_USER",
10    "isEnabled": true,
11    "isAccountNonExpired": true,
12    "isAccountNonLocked": true,
13    "isCredentailsNonExpired": true
14  }
15 }
```



www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

Update User

HTTP ECommerce_38 / user-service / Update User

PUT http://localhost:8085/api/users/2

Params Authorization Headers (9) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☒ JSON

```
1  {
2    "userId": 2,
3    "firstName": "rakesh1",
4    "lastName": "rakesh john",
5    "email": "rakesh1@gmail.com",
6    "phone": "9876543245",
7    "credential": {
8      "credentialId": 2,
9      "username": "rakesh",
10     "password": "123456",
11     "roleBasedAuthority": "ROLE_USER",
12     "isEnabled": true,
13     "isAccountNonExpired": true,
14     "isAccountNonLocked": true,
15     "isCredentialsNonExpired": true
16   }
```

Fetch User

GET http://localhost:8085/api/users/1

Params Authorization Headers (7) Body Scripts Settings

Query Params

	Key	Value
	Key	Value

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

Delete User

DELETE

⌵

http://localhost:8085/api/users/1

ParamsAuthorizationHeaders (7)BodyScriptsSettings

Query Params

	Key	Value
	Key	Value

Fetch Credentials



Ecommerce-Spring Boot / user-service / Credential / Fetch UserDetails

GET

⌵

http://localhost:8085/api/credentials/username/vijay

ParamsAuthorizationHeaders (7)BodyScriptsSettings

Query Params

	Key	Value
	Key	Value

Ant New Thov

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

Product Microservice

The **Product Microservice** is responsible for managing **products and categories** in the E-commerce application. It allows vendors to create, update, delete, and retrieve products and categories while ensuring **data consistency and modular design**.

The **Product Microservice** provides the following functionalities:

1. **Product Management**
 - Vendors can **create, update, delete, and retrieve** products.
 - Products belong to categories (**Many-to-One Relationship**).
2. **Category Management**
 - Categories support **subcategories** and **parent categories**.
 - Each category contains multiple products (**One-to-Many Relationship**).
 - CRUD operations for categories.
3. **DTO Pattern Implementation**
 - Uses **DTOs (Data Transfer Objects)** to:
 - **Expose only necessary data** in API responses.
 - **Structure request payloads efficiently**.

Database Relationships

Entity	Relationship Type	Related Entity	Mapped By	Join Column	JsonIgnore
Category	One-to-Many	Category (Self-referencing)	parentCategory	-	✓ (subCategories)
Category	Many-to-One	Category (Self-referencing)	-	parent_category_id	✗
Category	One-to-Many	Product	category	-	✓ (products)
Product	Many-to-One	Category	-	category_id	✗

Explanation:

- A Category can have multiple **subcategories**, forming a **self-referencing One-to-Many** relationship.
- A Category can have multiple **products** (One-to-Many).
- Each Product belongs to a **single** Category (Many-to-One).
- The parentCategory in Category establishes a **hierarchical structure**

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

Order Microservice

The **Order Microservice** is responsible for managing customer orders and cart operations in the **E-commerce Microservices Application**. It allows users to add items to their cart, place orders, and manage their orders efficiently. This service ensures data consistency using **Spring Boot**, **JPA**, and **DTO patterns**.

The **Order Microservice** provides the following functionalities:

- Users can place orders from their cart.
- Multiple orders can be linked to a **single cart** (One-to-Many relationship).
- Supports **CRUD operations** for orders.
- Stores details like order fee, date, and description.

Database Relationship



Entity	Relationship Type	Related Entity	Mapped By	Join Column	JsonIgnore
Cart	One-to-Many	Order	cart	-	✓ (orders)
Order	Many-to-One	Cart	-	cart_id	✗

Explanation:

- **Cart** has a **One-to-Many** relationship with **Order**. This means one **Cart** can have multiple **Orders**. The **orders** field in **Cart** is annotated with **@JsonIgnore** to prevent infinite recursion during serialization.
- **Order** has a **Many-to-One** relationship with **Cart**, meaning each **Order** belongs to one **Cart**. The **cart_id** is the foreign key in the **orders** table.

ght New Thoug

Payment Microservice

The **Payment Microservice** is responsible for processing payments for orders in the **E-commerce Microservices Application**. It ensures that transactions are securely processed and tracks the payment status for each order. This service integrates with the Order Microservice to verify order details before processing payments.

The **Payment Microservice** provides the following functionalities:

- **Process Payments:** Users can make payments for their orders.
- **Integration with Orders:** Links payments to specific orders.
- **CRUD Operations for Payments:** Allows retrieval and management of payment records.

Favourite Microservice

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

The **Favourite Microservice** is responsible for managing users' favourite products in the **E-commerce Microservices Application**. This service allows users to like or save products for future reference, helping enhance the shopping experience.

The **Favourite Microservice** provides the following functionalities:

- **Add Product to Favourites:** Users can mark products as favourite.
- **Retrieve Favourite Products:** Users can view a list of their saved products.
- **Remove Favourite Products:** Users can remove products from their favourite.
- **Track Liked Date:** Stores the date when a product was marked as favourite.

Eureka Server - Service Discovery for Microservices Architecture

The **Eureka Server** is a key component in a **microservices architecture** that enables **service discovery** and helps manage the dynamic registration and communication of all microservices. It ensures that services like **User Service, Product Service, Order Service, Payment Service, Favourite Service, Shipping Service, API Gateway, and Proxy Client** can discover and communicate with each other without hardcoding service locations.

1. Service Registration & Discovery

- Each microservice registers itself with Eureka, making its instance discoverable.
- When a service needs to communicate with another, it queries Eureka to get the latest instance address dynamically.

2. Load Balancing & Fault Tolerance

- Eureka enables **client-side load balancing** by distributing traffic among multiple instances of a service.
- If one service instance goes down, Eureka removes it from the registry, ensuring that traffic is only directed to healthy instances.

3. High Availability & Self-Healing

- Eureka continuously **checks the health** of registered services.
- If a service stops sending heartbeats, Eureka **deregisters** it to prevent failures.

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

Service	Function	Registered in Eureka?
User Service	Manages user authentication & profile	✓ Yes
Product Service	Handles product & category management	✓ Yes
Order Service	Manages orders and carts	✓ Yes
Payment Service	Handles payments and transactions	✓ Yes
Favourite Service	Manages users' favorite products	✓ Yes
Shipping Service	Handles order shipping and tracking	✓ Yes
API Gateway	Routes client requests to appropriate services	✓ Yes
Proxy Client	Secure authentication & communication	✓ Yes



Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1) (1)	UP (1) - api-gateway-deployment-5d64d4b467-qjzv:api-gateway:8767	
FAVOURITE-SERVICE	n/a (1) (1)	UP (1) - favourite-service-deployment-767ccb78cb-6m7vr:favourite-service:8089	
ORDER-SERVICE	n/a (1) (1)	UP (1) - order-service-deployment-7cb95f67b7-9ctrr:order-service:8086	
PAYMENT-SERVICE	n/a (1) (1)	UP (1) - payment-service-deployment-656998dd56-jbrzt:payment-service:8087	
PRODUCT-SERVICE	n/a (1) (1)	UP (1) - product-service-deployment-6cdb46b76d-q42rx:product-service:8067	
PROXY-CLIENT	n/a (1) (1)	UP (1) - proxy-client-deployment-845f8d7c85-5j7vr:proxy-client:8091	
SHIPPING-SERVICE	n/a (1) (1)	UP (1) - shipping-service-deployment-75f7c55f7d-h2p87:shipping-service:8090	
USER-SERVICE	n/a (2) (2)	UP (2) - user-service-deployment-6f5bbccb76-665w5:user-service:8085 , user-service-deployment-6f5bbccb76-7dwwc:user-service:8085	

Feign Client Communication for All Microservices

Feign is a declarative **HTTP client** in Spring Boot that allows microservices to communicate with each other **seamlessly** without manually handling REST API calls. It simplifies service-to-service communication by automatically integrating with Eureka Server for dynamic service discovery.

In our **E-commerce Microservices Application**, Feign clients are used to establish communication between:

- **User Service ↔ Order Service** (Fetching user details for order processing)
- **Order Service ↔ Product Service** (Fetching product details for orders)
- **Order Service ↔ Payment Service** (Validating payment status for orders)
- **Order Service ↔ Shipping Service** (Triggering shipping after successful payment)
- **Favourite Service ↔ Product Service** (Fetching product details for a user's favourite items)

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

- **API Gateway ↔ All Services** (Routing and authentication)

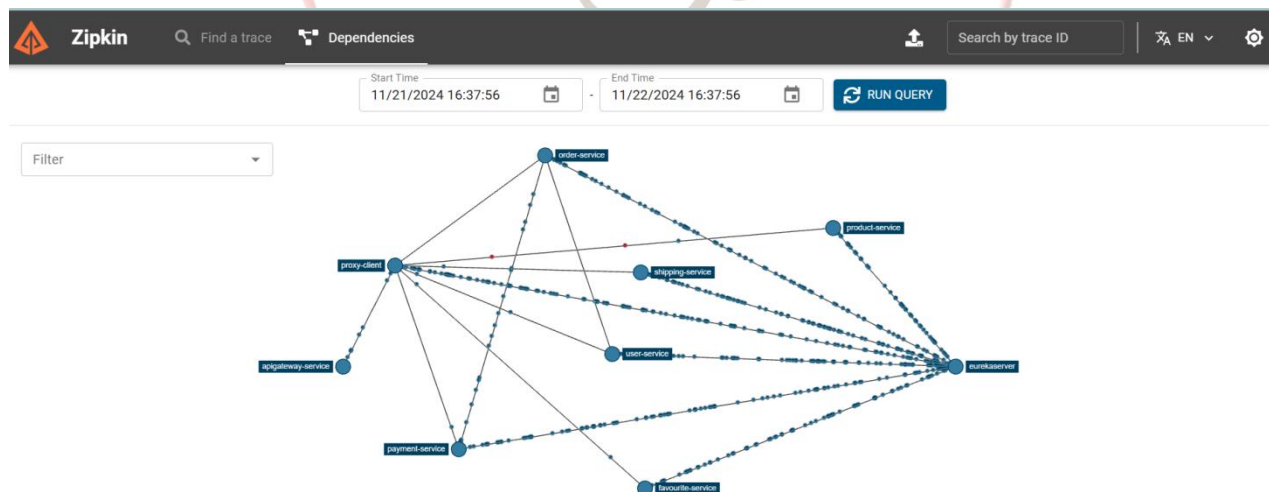
API Gateway for E-commerce Microservices

API Gateway acts as a **single entry point** for all client requests in the **E-commerce Microservices Architecture**. It handles:

- **Routing** requests to the appropriate microservices
- **Load balancing** between multiple instances of services
- **Security** through JWT authentication
- **Circuit breaking & resilience** with Resilience4j

In our project, the **Spring Cloud Gateway** is used as the API Gateway to manage requests across the following microservices:

- **User Service**
- **Product Service**
- **Order Service**
- **Payment Service**
- **Favourite Service**
- **Shipping Service**
- **Eureka Server** (Service Discovery)



Circuit Breaker Implementation in API Gateway

In microservices architecture, failures can happen at any time. If a service is **unavailable** or **slow**, it can cause cascading failures. To handle this, we use **Circuit Breaker** at the **API Gateway layer** to prevent overloading failing services and provide FallBack responses.

We implement Circuit Breaker using **Resilience4j** in **Spring Cloud Gateway** to:

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

- Detect failures and prevent repeated requests to a failing service.
- Provide a FallBack response if a microservice is down.
- Ensure high availability and resilience of the system.

Distributed Tracing with Zipkin in Microservices

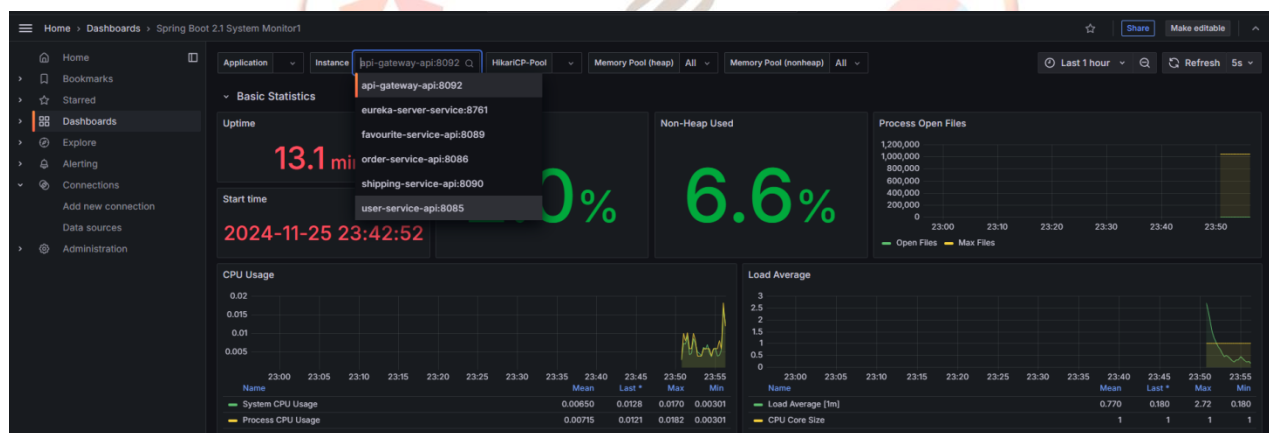
In microservices architecture, tracking requests across multiple services is **challenging**. **Distributed Tracing** helps to monitor and debug service-to-service communication.

- **Zipkin** is a **distributed tracing tool** that collects timing data and visualizes request flows across microservices.
- It helps **identify bottlenecks, performance issues, and failures** in a system.

Monitoring E-Commerce Microservices with Prometheus & Grafana

Monitoring is crucial in microservices architecture to ensure availability, detect issues, and optimize performance.

- **Prometheus**: Collects and stores real-time metrics from microservices.
- **Grafana**: Visualizes these metrics with dashboards for better insights.



Dockerizing Microservices for E-Commerce Application

Docker enables us to package each microservice as a lightweight, portable container. This ensures:

- ☐ **Consistency** – Works the same in development and production.
- ☐ **Scalability** – Can scale microservices independently.
- ☐ **Isolation** – Each microservice runs in its own container.

Each microservice will have its **own Dockerfile** and will be containerized separately.

Deploying Microservices on Kubernetes & Database Persistence

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

We will **deploy microservices on Kubernetes** by:

- **Containerizing services** and deploying them as **Pods**.
- Configuring **Kubernetes services** for networking.
- Setting up **Persistent Volume Claims (PVCs)** for database persistence.

Proxy Client Service

The **Proxy Client Service** in our **E-commerce Microservices Architecture** acts as an **intermediary** that:

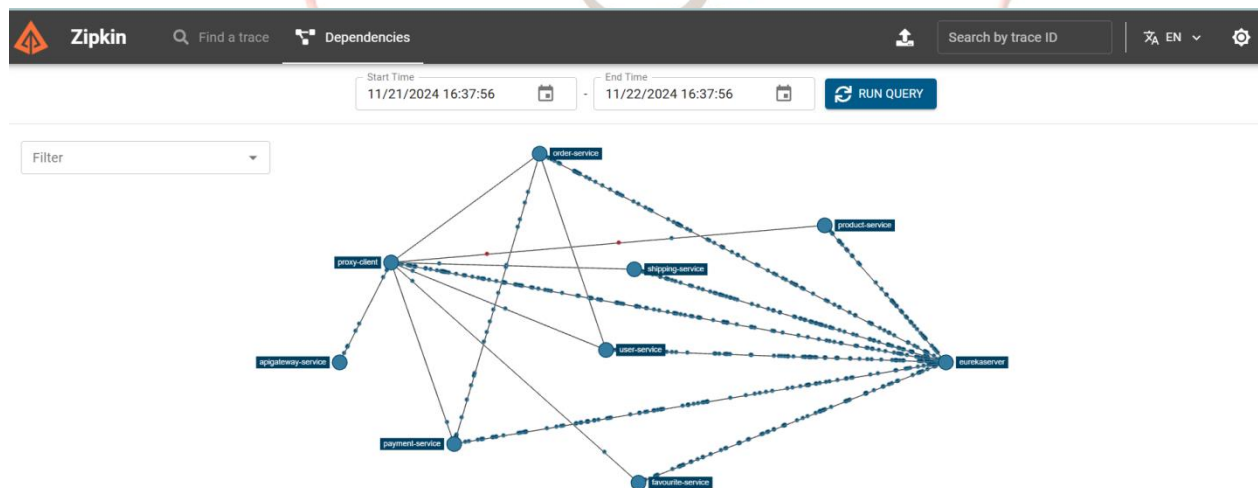
- Routes and manages requests between **microservices**.
- Uses **Feign Client** for **external API calls**.
- Implements **Role-Based Access Control (RBAC)** to **restrict access** based on user roles.
- Secures communication with **JWT-based authentication** at the **gateway level**

Final Project Review and Deployment

- Integrating all services for the complete e-commerce application

Project Screenshots

Zipkin Dashboard



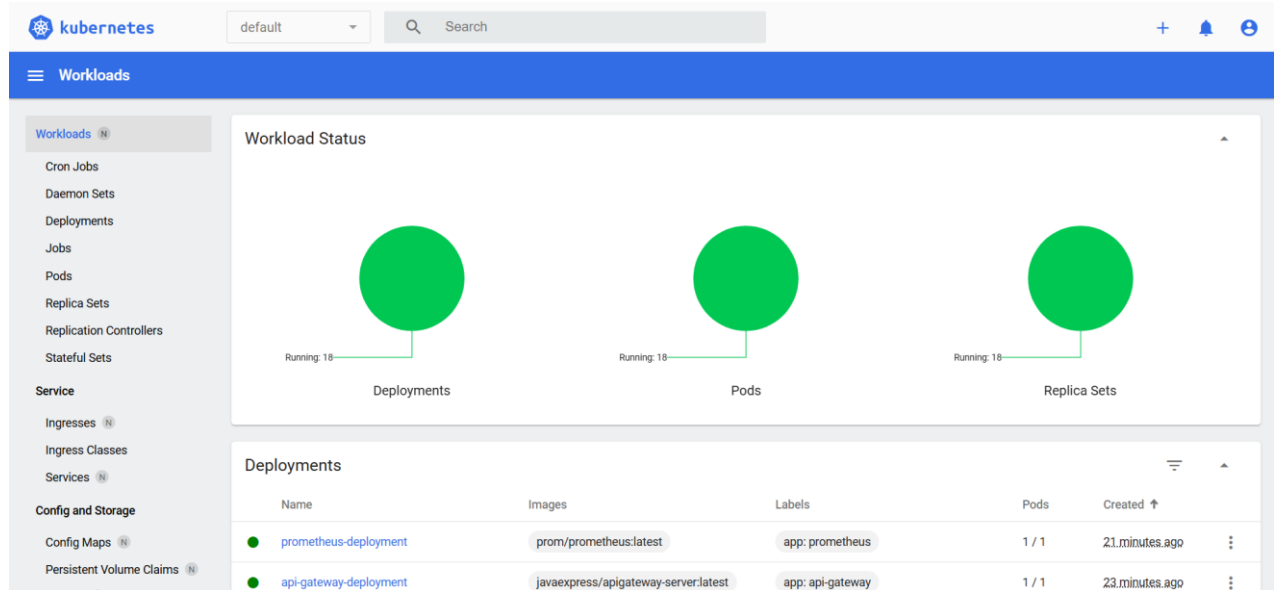
Kubernetes Dashboard

www.youtube.com/c/javaexpress

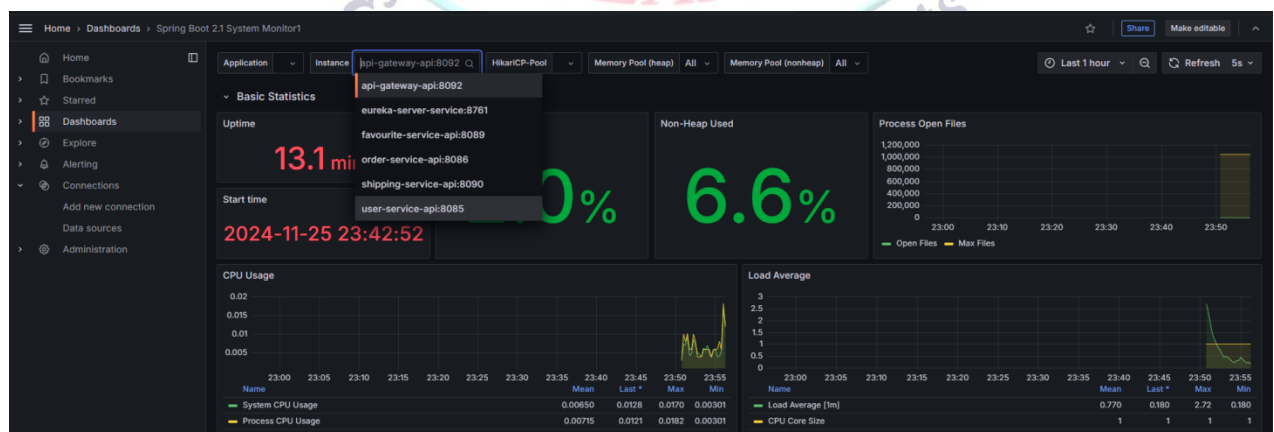
javaexpresschannel@gmail.com

+91 7801007910

Java Express



Grafana Dashboard



Kubernetes Deployments

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

NAME	READY	STATUS	RESTARTS	AGE
api-gateway-deployment-5d64d4b467-zp8hw	1/1	Running	0	40m
eureka-server-deployment-5847fd475d-fdqwb	1/1	Running	0	6h21m
favourite-database-deployment-6678d668c8-rr29r	1/1	Running	0	4h4m
favourite-service-deployment-767ccb78cb-6m7vr	1/1	Running	0	4h3m
grafana-deployment-7cf5f5b94b-dm85j	1/1	Running	0	20m
order-database-deployment-5dd7c69fd9-xn429	1/1	Running	0	4h53m
order-service-deployment-7cb95f67b7-9ctrr	1/1	Running	0	4h51m
payment-database-deployment-86d4bc4bb7-bcfbb	1/1	Running	0	4h14m
payment-service-deployment-656998dd56-jbrzt	1/1	Running	0	4h9m
product-database-deployment-774b7c4dcd-f9tr6	1/1	Running	0	4h30m
product-service-deployment-6cdb46b76d-q42rx	1/1	Running	0	4h29m
prometheus-deployment-647678c7bd-m8l22	1/1	Running	0	22m
proxy-client-deployment-845f8d7c85-5j7vr	1/1	Running	0	67m
shipping-database-deployment-5974c566cc-8hvjq	1/1	Running	0	4h
shipping-service-deployment-75f7c55f7d-h2p87	1/1	Running	0	3h59m
user-database-deployment-7cdb88bfdb-xb89m	1/1	Running	0	4h40m
user-service-deployment-6c6698cdb9-thshl	1/1	Running	0	14m
zipkin-deployment-84dcd4b776-bm45w	1/1	Running	1 (7h21m ago)	2d23h

Kubernetes Services

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
api-gateway-api	LoadBalancer	10.100.28.57	localhost	8092:31489/TCP	61m
eureka-server-service	LoadBalancer	10.103.107.49	localhost	8761:31336/TCP	2d23h
eureka-service	LoadBalancer	10.107.144.64	<pending>	8761:31078/TCP	7h17m
favourite-database-service	ClusterIP	None	<none>	3306/TCP	4h8m
favourite-service-api	LoadBalancer	10.104.157.208	localhost	8089:32356/TCP	4h7m
grafana-service	LoadBalancer	10.107.246.123	localhost	3000:31055/TCP	25m
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	59d
order-database-service	ClusterIP	None	<none>	3306/TCP	4h57m
order-service-api	LoadBalancer	10.105.31.184	localhost	8086:30427/TCP	5h9m
payment-database-service	ClusterIP	None	<none>	3306/TCP	4h18m
payment-service-api	LoadBalancer	10.96.185.120	localhost	8087:31557/TCP	4h18m
product-database-service	ClusterIP	None	<none>	3306/TCP	4h34m
product-service-api	LoadBalancer	10.111.130.77	localhost	8067:31228/TCP	5h43m
prometheus-service	LoadBalancer	10.104.40.103	localhost	9090:31601/TCP	26m
proxy-client-api	LoadBalancer	10.102.47.166	localhost	8091:30548/TCP	71m
shipping-database-service	ClusterIP	None	<none>	3306/TCP	4h4m
shipping-service-api	LoadBalancer	10.98.70.186	localhost	8090:31157/TCP	4h3m
user-database-service	ClusterIP	None	<none>	3306/TCP	4h44m
user-service-api	LoadBalancer	10.110.115.202	localhost	8085:30909/TCP	2d23h
zipkin-service	LoadBalancer	10.107.211.254	localhost	9411:30420/TCP	2d23h

Eureka Dashboard

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
APIGATEWAY-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.38:apigateway-service:8767
FAVOURITE-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.38:favourite-service:9090
ORDER-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.38:order-service:8086
PAYMENT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.38:payment-service:0
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.38:product-service
PROXY-CLIENT	n/a (1)	(1)	UP (1) - 192.168.1.38:PROXY-CLIENT:9093
SHIPPING-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.38:shipping-service:9091
USER-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.38:user-service:8085

www.youtube.com/c/javaexpress

javaexpresschannel@gmail.com

+91 7801007910

Java Express

Git Commands

```
1  git init .....> # Initialize repository
2
3  git clone <repo_url> .....> # Clone repository
4  git status .....> # View repository status
5
6  git add <file> .....> # Stage changes
7  git add . .....> # Stage all changes
8
9  git commit -m "message" .....> # Commit changes
10
11 git branch .....> # List branches
12 git branch <branch> .....> # Create branch
13 git checkout <branch> .....> # Switch to branch
14 git merge <branch> .....> # Merge branch
15
16 git remote -v .....> # View remotes
17 git pull origin <branch> .....> # Pull changes
18 git push origin <branch> .....> # Push changes
19
20 git log .....> # View commit history
21 git log --oneline .....> # View short commit history
22
23 git revert <commit> .....> # Revert commit
24 git reset <commit> .....> # Reset to commit
25
26 git tag .....> # List tags
27 git tag -a v1.0 -m "tag" .....> # Create annotated tag
28
29 git config --global user.name "name" .....> # Set global username
30 git config --global user.email "email" .....> # Set global email
31
```

Enlight New Thoughts